Implementing Tool-Calling Agents

https://github.com/dataville/simple_tool_calling_agent

Charles Rice

Senior Data Scientist

February 26th, 2026

About Me

Background

- Data Scientist, 5+ years
- Software Engineer, 25+ years
- B.S. Mathematics
- Graduate coursework and research: neural networks, evolutionary computation, ML, AI, RL

Selected Projects

- Researched evolving analog circuits for spacecraft self repair (evolutionary computation)
- 3D body scan analysis for clothing size determination
- Automated electron microscope image acquisition
- Email engagement classifier: human vs. spam filter interaction
- AI agent systems: email sequencing, customer segmentation, BaristaBot, medical resource advisor

Scope

What This Talk Covers

- What problems tools and agents solve
- Nine implementation requirements:
  - Requirements 1-7: functional - missing any breaks tool calling
  - Requirements 8-9: observability - system runs, but you won't know if it works

1. Model selection
2. Pydantic classes
3. Tool decorator
4. Docstrings
5. Tool binding
6. System instructions
7. Graph construction
8. Verify execution
9. Evaluation

What This Talk Does NOT Cover

- Deployment strategies
- Scaling to production
- Advanced agent patterns
- Multi-agent systems
- Security hardening
- Production monitoring infrastructure

What Is an Agent?

An agent is a system that:

- Uses an LLM to decide what to do next
- Executes actions (tools)
- Observes results
- Repeats until a stopping condition is met
- It is a loop: Think → Act → Observe → Repeat.

What an Agent Framework Provides

An agent framework provides:

- A structured execution loop
- Deterministic routing between steps
- State management (message history)
- Tool execution and error handling
- Termination control

It does NOT provide:

- Correct reasoning
- Guaranteed tool selection
- Guaranteed correctness

LLMs Are Non-Deterministic

- LLMs are non deterministic — the same input can produce different outputs
- This produces hallucinations, inconsistent and incorrect responses

The Role of the LLM in a Tool-Calling Agent

The LLM does three things:

1. Interprets user intent
2. Decides whether a tool is needed
3. Extracts structured arguments for the tool

The LLM does NOT:

- Execute code
- Validate arguments
- Control routing
- Maintain state

Those are handled by the framework.

What is a Tool?

- A tool is a Python function you write and register so the LLM can call it
- Not a black box — your code, your logic, your external calls
- The LLM decides when to call it and what arguments to pass — your function does the actual work
- You have to document your functions as you write them
- Tools produce actual verified data

```
@tool(args_schema=WeatherInput)
def get_weather(location: str) -> str:
    """Retrieves current weather
    for a given location."""
    return call_weather_api(location)
```

What Tools Solve: External Systems

LLMs are isolated from all external systems. They cannot read current data from them and cannot write to or trigger actions in them. Same root cause — two distinct failure modes.

WITHOUT Tool Calling

```
READ — no access to current data:
User: "What's the weather in Boston?"
LLM: "It is currently 43°F and raining."
[Specific, confident, invented]

WRITE — cannot affect external systems:
User: "Add Jane Smith as a lead in the CRM."
LLM: "Done! Jane Smith has been added."
[Claims success - nothing happened]
```

WITH Tool Calling

```
READ — actual data returned:
get_weather("Boston") -> "72°F, sunny"
LLM: "Current temperature in Boston is
72°F."
[From actual function execution]

WRITE — state change executed:
create_crm_lead("Jane Smith") -> "Lead ID
#4821"
LLM: "Jane Smith has been added as lead
#4821."
[Confirmed by ToolMessage - record
exists]
```

Reads return verified data. Writes confirm state change. Neither is possible without tools.

What Tools Solve: The Computation Problem

Limitation 2 — illustrated here: LLMs do not compute, they predict. For simple arithmetic they often predict correctly because the answer appeared in training. For less common calculations they may predict a plausible but wrong number.
The dangerous failure mode: a confident, specific, wrong answer with no warning in the output.

WITHOUT Tool Calling - Same Query, Three Different Failures

```
User: "What is 14% of $847?"

LLM (training pattern): "To find 14% of $847, multiply $847 by
0.14. The answer is $118.58."
[Happens to be correct - but was it computed or predicted?]

LLM (hallucination): "14% of $847 is $121.48."
[Specific, confident, wrong - no warning in the output]

LLM (describes action): "I would use a calculator to compute 14%
of $847."
[Description - nothing executed]
```

WITH Tool Calling

```
User: "What is 14% of $847?"

LLM: "14% of $847 is $118.58."
[847 * 0.14 = 118.58 - computed by the
calculator tool, not predicted]
```

Tool calling grounds the response in executed code — not in predicted text.

Messages

A message is a Python object representing one turn in the system.
Examples: HumanMessage, AIMessage, ToolMessage.
The graph passes a growing list of these messages between nodes.
The entire agent state is just a list of these messages.

`HumanMessage` — The user input. Created when you invoke the graph.

`AIMessage` — The LLM response. May contain tool_calls (structured JSON) or a natural language response.

`ToolMessage` — The tool result. Created by ToolNode after executing the Python function. Contains the function return value and the tool call ID it answers.

All three are appended to MessagesState in order. The full list is the agent conversation history.

Key Insight

The LLM never sees Python objects directly — it only sees their serialized message and JSON representations. Every interaction happens through typed messages passed in the conversation history.

Implementation Requirements

Requirement 1 - Model Selection

Function calling is a trained capability - the model must be specifically trained to generate structured tool calls, format JSON correctly, and know when to invoke a tool versus answer directly. Wrong model choice produces multiple failure modes: text descriptions instead of structured calls, hallucinated answers with no tool call attempt, or malformed JSON that fails parsing. Everything downstream breaks.

Function-Calling Capable

Foundation models (commercial APIs):

- GPT-4 and newer OpenAI models
- Claude 3+ models

Small/open-source (self-hosted):

- Hermes-3-Llama-3.1-8B (used in this notebook)
- Llama 3.1-Instruct, 3.2-Instruct, 3.3-Instruct
- Qwen2.5-Instruct
- Mixtral-8x7B-Instruct
- Granite-3.1-8B-Instruct

Lacks Function Calling

- Base models without instruction tuning (e.g., Llama-3.1-8B)
- Older instruct models (e.g., Mistral-7B-Instruct-v0.1)

Requirement 2 - Pydantic Classes

Python Code

```
from pydantic import BaseModel, Field

class WeatherInput(BaseModel):
    location: str = Field(
        description="The city and optionally state/
country (e.g., 'San Francisco, CA')",
        min_length=2,
        max_length=100
    )
```

Generated JSON Schema (sent to LLM)

```
{
  "properties": {
    "location": {
      "description": "...",
      "maxLength": 100,
      "minLength": 2,
      "title": "Location",
      "type": "string"
    }
  },
  "required": ["location"],
  "title": "WeatherInput",
  "type": "object"
}
```

What This Provides

- Type safety (str, float, int, etc.)
- Validation (min/max length, numerical ranges)
- Schema generation: Pydantic generates JSON schemas that LangChain sends to the LLM via tool binding
- Field descriptions (guide LLM behavior)

Requirement 3 - Tool Decorator

```
@tool(args_schema=WeatherInput)
def get_weather(location: str) -> str:
    """
    Retrieves current weather information for a
specified location.

    Args:
        location: The city and optionally state/country

    Returns:
        A string containing current temperature and
conditions.
    """
    return f"Current weather in {location}: 72 degrees
F, sunny"
```

What This Does

- The decorator converts the function into a Tool object (name, docstring, args schema, callable).
- It does not execute the function.
- It does not register the tool
- The agent does not know about the tool yet.

Requirement 4 - Docstrings

```
@tool(args_schema=WeatherInput)
def get_weather(location: str) -> str:
    """
    Retrieves current weather information for a
specified location.

    Args:
        location: The city and optionally state/country

    Returns:
        A string containing current temperature and
conditions.
    """
```

Critical Point

Docstrings are the PRIMARY mechanism by which the LLM learns what each tool does. The docstring content is sent to the LLM as part of the tool schema via tool binding.

What Makes Good Docstrings

- Clear purpose statement
- Detailed parameter descriptions with examples
- Return value documentation
- Format and constraints

Debugging Tool Calls

If your tool isn't being called correctly or consistently, examine the docstring first. Small changes have large effects on tool selection and parameter accuracy.

Requirement 5 - Tool Binding

```
# Base LLM - NO tools
llm = ChatOpenAI(
    base_url="http://localhost:8082/v1",
    model="NousResearch/Hermes-3-Llama-3.1-8B",
    temperature=0  # ALWAYS set explicitly
)

# Agent LLM - WITH tools bound
tools = [get_weather, calculator]
llm_with_tools = llm.bind_tools(tools)
```

Binding Is A Critical Step That Many Coding Agents Skip

What Binding Does

- bind_tools(tools) returns an LLM instance configured so that tool definitions are included in future model calls.
- Without binding, tool definitions are not sent, so the model will answer in text or hallucinate because it was never given the tool interface.
- With binding, the model has the tool specs at inference time and can emit structured tool_calls that match the schema.

Note: The docstring becomes the "description" field. The Pydantic Field descriptions become parameter descriptions.

What the LLM Actually Receives (get_weather shown)

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Retrieves current weather
information for a specified location.
Args:
    location: The city name and optionally
    state/country
Returns: A string containing the current
    temperature and weather conditions.",
    "parameters": {
      "properties": {
        "location": {
          "description": "The city name and
state/country (e.g., 'San Francisco, CA')",
          "maxLength": 100, "minLength": 2,
          "type": "string"
        }
      },
      "required": ["location"],
      "type": "object"
    }
  }
}
```

LLM Configuration - Don't Rely on Defaults

```
llm = ChatOpenAI(
    base_url="http://localhost:8082/v1",
    api_key="not-needed",
    model="NousResearch/Hermes-3-Llama-3.1-8B",
    temperature=0  # ALWAYS set explicitly - do not rely on defaults
)
```

- temperature is set at instantiation - not at binding
- temperature controls sampling randomness (range: 0.0-2.0)
- ALWAYS set temperature=0 explicitly for tool calling
- Do not rely on defaults - different servers apply different defaults (OpenAI API: 1.0)
- Tool calling requires consistency, not creativity: consistent tool selection, consistent parameter extraction, consistent JSON formatting
- Temperature=0 reduces variance but does not make LLM behavior deterministic.
- Set temperature=0 on both the agent LLM and the evaluator LLM

This is a configuration that affects every LLM call in the system.

Requirement 6 - System Instructions

```
system_message = """You are a helpful assistant with access to specific tools.

You MUST call a tool to answer questions. If no tool is available for the
user's question, respond with: 'I don't have a tool to answer that question.'"""
```

What System Instructions Provide

- Behavioral guidelines
- Tool usage enforcement
- Fallback behavior
- NOT tool information (already in schemas)

Common Mistakes

- AI assistants often list tools in system message (maintenance nightmare, increases context, confuses LLM)
- AI coding assistants often skip system message entirely
- Second place to check if tools aren't being called properly (after docstrings)

Note: System instructions guide LLM behavior - they do not guarantee it

Requirement 7 - Graph Construction

```
graph_builder = StateGraph(MessagesState)
graph_builder.add_node("agent", call_model)
graph_builder.add_node("tools", ToolNode(tools))
graph_builder.add_conditional_edges(
    "agent",
    lambda x: "tools" if x["messages"][-1].tool_calls else END
)
graph_builder.add_edge("tools", "agent")
graph_builder.add_edge(START, "agent")
graph = graph_builder.compile()
```

Flow: START -> agent -> (conditional) -> tools -> agent -> (conditional) -> END

Key Insight

Graph structure is deterministic - LLM decisions within it are not

How Tool Execution Works

1. Agent node runs (LLM call). Appends an AIMessage (may include tool_calls).
2. Framework evaluates last message.
   - If tool_calls → route to ToolNode
   - Else → END
3. ToolNode validates args and executes each tool call. Appends one ToolMessage per call.
4. Framework routes back to agent node.
5. Agent node runs again with full message history
6. Framework evaluates again (loop or END)

Key Insight

The conditional edge fires twice per tool-using query: once to dispatch, once to exit.

How Validation Errors Propagate

Scenario 1: Successful Tool Call

```
HumanMessage: "What's the weather in
Boston?"
AIMessage: tool_calls=[{'name':
'get_weather',
  'args': {'location': 'Boston'}}]
LLM correctly extracted location

ToolMessage: "Current weather in Boston:
72F, sunny"
Tool executed successfully

AIMessage: "The current temperature in
Boston
is 72F and sunny."
```

Scenario 2: Self-Correction With Available Context

```
HumanMessage: "What's the weather in
Boston?"
AIMessage: tool_calls=[{'name':
'get_weather', 'args': {}}]
LLM forgot to extract location

ToolMessage: "Error: ValidationError:
location
Field required [type=missing]"
ToolNode caught error, returned as
ToolMessage

AIMessage: tool_calls=[{'location':
'Boston'}]
LLM corrected its call

ToolMessage: "Current weather in Boston:
72F, sunny"
AIMessage: "The current temp in Boston is
72F."
```

Scenario 3: Missing Information

```
HumanMessage: "What's the weather?"
AIMessage: tool_calls=[{'name':
'get_weather',
  'args': {}}]
LLM has no location to extract

ToolMessage: "Error: ValidationError:
location Field required [type=missing]"

AIMessage: "I need a location to check
the
weather. Which city would you like
weather
information for?"
LLM asks user for missing information
```

Note: These scenarios represent typical behavior, not guaranteed outcomes. The architecture constrains the failure modes - it does not eliminate them.

Requirement 8 - Verify Execution

```
# Run the agent
result = graph.invoke(
    {"messages": [("user", question)]}
)

# Programmatic verification
# Did ANY tool execute?
has_tool_message = any(
    isinstance(msg, ToolMessage)
    for msg in result["messages"]
)

if has_tool_message:
    print("Tool executed")
else:
    print("No tool execution")
```

Message Flow Example

```
HumanMessage: "What's the weather in Boston?"
AIMessage: tool_calls=[{'name': 'get_weather',
    'args': {'location': 'Boston'}}]
ToolMessage: "Current weather in Boston: 72F,
sunny"
AIMessage: "The current temperature in
Boston is 72F, sunny."
```

This is BASIC verification only.

Complete Evaluation Is Broader

- Right tool called?
- Correct parameters?
- Output correct?

Requirement 9 - Evaluation

```
eval_prompt = f"""You are evaluating an AI agent's
performance.

USER QUESTION: {question}
TOOLS USED BY AGENT: {tools_used}
AGENT'S FINAL ANSWER: {final_answer}

Evaluate on these criteria:
TOOL_SELECTION: correct/incorrect/missing
QUALITY: good/fair/poor
OVERALL: pass/fail
"""

# Use raw LLM - no tools bound
eval_response = llm.invoke(eval_prompt)
```

Why LLM-Based Evaluation

Keyword matching breaks on rephrasing: "multiply" vs "multiplied" vs "times"
LLM understands semantic equivalence
Returns structured, reasoned assessment

Production Architecture

Use a separate model for evaluation - the same model evaluating its own output introduces bias; it will tend to rate its own responses favorably

This Notebook

Single model used in two modes due to 24GB GPU memory constraint - this is a resource workaround, not a recommended architecture

Critical Point: Use llm (base), not llm_with_tools - the evaluator must not have access to tools

Summary

Why Agents

- LLMs have no knowledge beyond their training data
- Agents provide MORE deterministic behavior through constraints
- Tool calling: verifiable actions vs. free-form text

Nine Requirements

- Model selection, Pydantic, tool decorator, docstrings
- Tool binding, system instructions, graph construction
- Verification, evaluation
- Requirements 1-7: functional - miss any and tool calling breaks
- Requirements 8-9: observability - without these, you can’t tell if it’s working

Observations

- Implementation is a Software Engineering problem, not a Data Science problem

Foundation Matters

Questions?